

Tytuł projektu: 25 WYKORZYSTANIE DUŻYCH MODELI JĘZYKOWYCH (LLM) DO
AUTOMATYCZNEGO
GENEROWANIA I ANALIZY GRAFÓW INFRASTRUKTURY LINIOWEJ Z UŻYCIEM BAZ
GRAFOWYCH

Autorzy: Mateusz Młynek, Michał Szczepanik.

Przedmiot: Hurtownie Danych i Systemy Eksploracji
Danych

Grupa: Grupa nr. 1 (usos), BDILS-1

Data: 25.05.2026 r.

Cel projektu:

Stworzenie prototypu systemu, który konwertuje nieustrukturyzowane opisy tekstowe na ustrukturyzowany format grafowy. Projekt miał na celu ocenę zdolności modeli LLM do identyfikacji obiektów technicznych (np. mostów, skrzyżowań) oraz implementację bazy Neo4j pozwalającej na zaawansowaną analizę topologiczną.

Rozwiązany problem: Tradycyjne metody modelowania infrastruktury liniowej opierają się na ręcznej digitalizacji. Proces ten jest bardzo czasochłonny, wysoce kosztowny oraz obciążony dużym ryzykiem błędów ludzkich.

Zakres wykonanych prac:

- Przygotowanie listy obiektów, relacji oraz atrybutów które będzie musiał definiować model
- Przygotowanie złotego standardu z realnych tekstów z którymi będzie porównywany model
- Opracowanie architektury potoku danych (Data Pipeline).
- Przygotowanie inżynierii promptów, w tym rygorystycznych instrukcji systemowych (System Prompt) i wzorców metodą Few-Shot Learning, aby zmusić model do zwracania czystego kodu JSON.
- Wykorzystanie API modelu Gemini do wyodrębniania encji i relacji z surowych tekstów.
- Implementacja skryptu ładującego JSON do bazy Neo4j przy wykorzystaniu języka zapytań Cypher.
- Przeprowadzenie eksperymentów analitycznych, w tym ewaluacji metryk (Recall, Precision, F1-Score) oraz badanie wpływu temperatury modelu na jakość i stabilność wyników (zjawisko fuzji wiedzy i tzw. "wysp danych").

Dane wejściowe

Pochodzenie: Surowe, nieustrukturyzowane dane tekstowe pochodziły z publicznych komunikatów prasowych i aktualizacji technicznych publikowanych przez Generalną Dyрекcję Dróg Krajowych i Autostrad (GDDKiA).

Format: Dokumenty tekstowe (tekst naturalny).

Zawartość: Opisy planowanych modernizacji, budowy obwodnic, a także analizy infrastrukturalne dróg (m.in. rozbudowa autostrady A4, węzeł Krzyszkowice na DK7. Teksty zawierały parametry inżynierskie dróg, obiektów i planowanych relacji przestrzennych.

Teksty na których były przeprowadzane próby:

<https://www.gov.pl/web/gddkia/kompleksowe-zmiany-na-a4>

<https://www.gov.pl/web/gddkia-krakow/wezel-krajowy-drogi-lokalne>

Założenia projektowe:

-Na początku prób model był testowany zarówno w trybie stochastyczny jak i deterministycznym, w dalszej części prób przyjęto w pełni deterministyczny tryb pracy modelu LLM (temperatura $T=0.0$) – było to uznane za warunek konieczny z punktu widzenia inżynierskiego dla zachowania stabilności strukturalnej plików JSON oraz zapewnienia powtarzalności wyników.

-Założono wykorzystanie konkretnej, sztywnej ontologii (słownika), definiującej dozwolone węzły (Odcinek drogi, Skrzyżowanie, Rondo, Obiekt inżynierski, Punkt POI) oraz konkretne relacje pomiędzy nimi (np. ŁĄCZY_SIE_Z, ZAWIERA).

- Pominęto atrybuty, o których brakowało informacji w tekście źródłowym.

-Integracja z bazą grafową neo4j

Opis rozwiązania

Działanie systemu: W procesie konwersji niesstrukturyzowanych komunikatów technicznych na model grafowy wykorzystano model **Gemini 2.5 Flash** za pośrednictwem oficjalnego SDK google-genai.

Proces projektowy oparto na następujących paradygmatach inżynierii promptów:

Rygorystyczne Instrukcje Systemowe (Role Prompting): Model został osadzony w roli eksperta ds. inżynierii drogowej oraz systemów bazodanowych. Wymusiło to stosowanie właściwej terminologii technicznej (klasyfikacja dróg, typy skrzyżowań i relacje inkluzji). *

Kontekstowe uczenie kilkukrotne (Few-Shot Learning): W celu zminimalizowania ryzyka halucynacji semantycznych i syntaktycznych, w prompt systemowy wbudowano pełny, rzeczywisty przykład mapowania. Wzorzec ten jasno zdefiniował oczekiwaną relację między encjami i strukturę właściwości. **Wymuszenie Determinizmu Strukturalnego:**

Kluczowym krokiem inżynierskim było zablokowanie probabilistycznego charakteru modelu. Zastosowano parametr temperatury **$T=0.0$** oraz programowe wymuszenie formatu odpowiedzi `response_mime_type="application/json"`. W badaniach pilotażowych przy $T>0.0$ model generował tzw. *szum tokenizacji*.

Architektura i narzędzia: Całość logiki biznesowej oparto na języku Python 3.10+ (integrując biblioteki google-genai oraz neo4j Python Driver). Do przesyłania poleceń do bazy zastosowano zapytania Cypher

Model danych / baza danych (Grafowa)

Struktura została sprowadzona do grafu skierowanego:

Węzły:

- Odcinek drogi (atributy: nawierzchnia, długość, liczba pasów, klasa drogi).
- Skrzyżowanie (atributy: rodzaj, sygnalizacja świetlna).
- Rondo (atributy: liczba wlotów, nazwa, lokalizacja).
- Obiekt inżynierski (typ: Most/Wiadukt/Tunel, nośność, opis).
- Punkt POI (kategoria np. MOP/Stacja, nazwa własna).

Relacje:

- ŁĄCZY_SIE_Z: Pomiedzy Odcinkiem a Skrzyżowaniem/Rondem.
- ZAWIERA: Odcinek zawiera Obiekt inżynieryjny lub rondo.
- PRZECHODZI_NAD/POD: Obiekt inżynieryjny <-> droga / przeszkoda.
- DOJAZD_DO / PRZY_DRODZE: Połączenie odcinka z usługami POI.

Implementacja/walidacja

Opis kodu i podziału na moduły

Całość logiki biznesowej została zamknięta w jednym, spójnym skrypcie wykonywalnym w języku Python. Program realizuje proces liniowo (potokowo) i dzieli się na dwa główne moduły logiczne:

1. **Moduł Ekstrakcji Semantycznej (extract_with_gemini):** Odpowiada za wczytanie surowego tekstu, zapętlenie zapytania z promptem systemowym, wymuszenie determinizmu (temperatura T=0.0) oraz odebranie ustrukturyzowanego dokumentu JSON. Wynik pośredni jest automatycznie zrzucany do lokalnego pliku ostatni_wynik.json w celu zachowania śladu danych.
2. **Moduł Integracji i Audytu Grafów (save_to_neo4j):** Odpowiada za otwarcie sesji z bazą danych w chmurze (Neo4j Aura), automatyczne stworzenie obiektów (węzłów), a następnie uruchomienie autorskiej pętli sprawdzającej relacje z oficjalną ontologią przed wykonaniem ostatecznego zapisu.

Biblioteki standardowe (wbudowane w Pythona):

- **json:** Służy do parsowania tekstowych odpowiedzi z Gemini na słowniki Pythona, formatowania danych oraz zapisu wyników do pliku **.json**.
- **os:** Odpowiada za operacje na systemie plików – tworzenie folderów, sprawdzanie czy pliki istnieją oraz listowanie zawartości katalogu.

Biblioteki zewnętrzne (wymagające instalacji):

- **google-genai** (from google import genai): Oficjalny, nowoczesny SDK od Google do komunikacji z modelami Gemini. Wykorzystuje parametr **response_mime_type="application/json"**, aby zagwarantować stabilną strukturę wyjściową.
- **neo4j:** Oficjalny sterownik (*driver*) do bazy danych Neo4j. Odpowiada za nawiązanie bezpiecznego połączenia i wykonywanie zapytań w języku **Cypher**

Instrukcja Uruchomienia i Konfiguracja (Config)

1. Przygotowanie środowiska wirtualnego (Virtual Env)

W głównym katalogu projektu utwórz i aktywuj wirtualne środowisko Pythona, a następnie zainstaluj wymagane pakiety zewnętrzne zdefiniowane w pliku manifestu:

Bash

Tworzenie środowiska wirtualnego

```
python -m venv .venv
```

Aktywacja środowiska (System Windows)

```
.venv\Scripts\activate
```

Aktywacja środowiska (Systemy Linux / macOS)

```
source .venv/bin/activate
```

Instalacja wymaganych bibliotek z pliku requirements.txt

```
pip install -r requirements.txt
```

2. Konfiguracja zmiennych środowiskowych (.env)

Dane dostępowe do chmurowej bazy danych oraz klucze API zostały odizolowane od kodu źródłowego ze względów bezpieczeństwa.

Skopiuj plik demonstracyjny .env.example i utwórz na jego podstawie lokalny plik konfiguracyjny .env:

Bash

```
cp .env.example .env
```

Otwórz nowo utworzony plik .env i wprowadź swoje tajne klucze autoryzacyjne:

```
NEO4J_URI=neo4j+s://twoj-id.databases.neo4j.io
```

```
NEO4J_USER=neo4j
```

```
NEO4J_PASSWORD=twoje-haslo-do-bazy
```

```
GEMINI_API_KEY=twoj-klucz-api-gemini
```

3. Zasilenie danymi i uruchomienie programu

Upewnij się, że w katalogu głównym projektu znajduje się folder data. Jeśli katalog nie istnieje, program utworzy go automatycznie przy pierwszym uruchomieniu.

Umieść surowe pliki tekstowe (z rozszerzeniem .txt) zawierające opisy techniczne dróg w folderze data.

Uruchom główny skrypt aplikacji:

Bash

```
python nazwa_twojego_skryptu.py
```

Po wyświetleniu interaktywnego menu w konsoli, wpisz numery plików oddzielone spacją (np. 1 2 5), aby rozpocząć ekstrakcję danych przez LLM i automatyczne zasilenie bazy Neo4j.

Wizualizacja i Przegląd Wyników w Neo4j Browser

Aby zweryfikować poprawność zaimportowanych danych i wyświetlić wygenerowany graf infrastruktury liniowej, zaloguj się do panelu Neo4j Aura / Neo4j Browser i użyj poniższych zapytań w języku Cypher:

1. Wyświetlenie całego wygenerowanego grafu

To podstawowe zapytanie, które pozwoli zobaczyć pełną sieć powiązań (wszystkie pobrane węzły oraz relacje przestrzenne między nimi):

Cypher

```
MATCH (n) OPTIONAL MATCH (n)-[r]->(m) RETURN n, r, m
```

2. Czyszczenie bazy danych (Reset przed kolejnym testem)

Aby usunąć wszystkie dane z bazy w celu przetestowania kolejnego grafu, należy przed wgraniem nowych danych wykonać zapytanie czyszczące:

Cypher

```
MATCH (n) DETACH DELETE n
```

Walidacja poprawności:

W finalnej architekturze potoku danych zdecydowano się na zaimplementowanie **modułu pasywnego audytu semantycznego (Soft Validation)** zamiast restrykcyjnego blokowania zapytań Cypher. Zmiana ta była podyktowana kluczowymi paradygmatami inżynierii systemów opartych na modelach LLM:

1. Realizacja paradygmatu Human-in-the-Loop: Zgodnie z założeniami wdrożeniowymi, automatycznie generowany graf pełni funkcję „wersji kandydackiej” (szkicu), która

docelowo podlega weryfikacji przez inżyniera kontraktu. Twarde blokowanie relacji przez skrypt Pythona odbierałoby ekspertowi możliwość oceny intencji modelu LLM.

2. **Ochrona przed utratą wiedzy (Knowledge Enrichment):** Anomalie relacyjne generowane przez Gemini (np. użycie ZAWIERA pomiędzy dwoma odcinkami dróg w celu oddania ich hierarchii) często niosą ze sobą poprawną intuicję logiczną i inżynierską, mimo naruszenia sztywnych ram przyjętej ontologii. Zastosowanie miękkiej walidacji pozwala na pełny import topologii do bazy Neo4j, zapobiegając bezpowrotnej utracie danych technicznych.

3. **Cele diagnostyczno-analityczne:** Tryb ostrzegawczy umożliwia transparentne monitorowanie zachowań modelu bezpośrednio w logach konsoli. System precyzyjnie wskazuje i flaguje anomalie (np. próby łączenia dróg z obiektami mostowymi za pomocą relacji ŁĄCZY_SIE_Z), dostarczając twardych metryk do analizy skuteczności inżynierii promptów bez przerywania ciągłości zasilania hurtowni danych

Program przetestowano metodą wielokrotnych, empirycznych prób przy różnej temperaturze modelu. Testy wykazały, że przy $T>0.0$ pojawiał się "szum tokenizacji" (np. błędy w nazwie relacji jak ŁĄCZY_SIIĘ_Z), który powodował wywrotki bazy danych. Wyciągnięto wniosek, że LLM nie jest systemem w 100% samowystarczalnym.

Model danych i schemat bazy grafowej

W projekcie zrezygnowano z tradycyjnego modelu tabelarycznego na rzecz **modelu grafu właściwości**. W tym podejściu dane nie są dzielone na sztywne tabele, lecz reprezentowane jako sieć połączonych obiektów. Schemat bazy składa się z trzech filarów: **Etykiet (Labels)**, **Właściwości (Properties)** oraz **Relacji (Relationships)**.

Węzły reprezentują fizyczne i logiczne obiekty infrastruktury drogowej. Każdy węzeł posiada unikalny identyfikator techniczny (np. N1, N2) oraz zestaw cech opisowych.

Etykieta węzła	Opis obiektu	Kluczowe atrybuty (Właściwości)
Odcinek drogi	Podstawowy element sieci liniowej	nazwa, nawierzchnia, długość (km), liczba pasów, klasa drogi (np. GP, S, A)
Skrzyżowanie	Punkt styku co najmniej dwóch dróg	nazwa, rodzaj (np. węzeł, T-kształtne), sygnalizacja (Tak/Nie)
Rondo	Specyficzny typ skrzyżowania kołowego	nazwa, liczba wlotów, lokalizacja
Obiekt inżynierski	Budowla w ciągu drogi	typ (Most, Wiadukt, Tunel, Przepust), nazwa, nośność (t)
Punkt POI	Obiekt użyteczności przy drodze	kategoria (np. MOP, Stacja, Parking), nazwa własna

Relacje w grafie są skierowane i niosą ze sobą konkretne znaczenie semantyczne. Definiują one topologię sieci drogowej oraz hierarchię obiektów.

- **ŁĄCZY_SIE_Z**: Relacja topologiczna. Łączy Odcinek drogi z punktami węzłowymi (Skrzyżowanie lub Rondo). Jest to kluczowe dla algorytmów wyznaczania tras.
- **ZAWIERA**: Relacja hierarchiczna (inkluzja). Wskazuje, że dany Odcinek drogi posiada w swoim ciągu konkretny Obiekt inżynierski lub Rondo.
- **PRZECHODZI_NAD / PRZECHODZI_POD**: Relacje przestrzenne. Pozwalają na modelowanie wielopoziomowych węzłów, określając, który Obiekt inżynierski znajduje się nad lub pod daną trasą.
- **DOJAZD_DO**: Relacja usługowa. Łączy Odcinek drogi lub Obiekt bezpośrednio z punktem POI.

Analiza wyników

Eksperymenty integracyjne (jednoczesne przetwarzanie połączonych tekstów dla Autostrady A4 oraz Zakopianki DK7) pozwoliły na zbadanie dwóch kluczowych zjawisk z zakresu integracji danych:

A. Autonomiczna Rezolucja Encji (Entity Resolution)

W scenariuszu udanym (Próby nr 1 i nr 3), model wykazał się wysoką zdolnością do tzw. uzgadniania encji. Pomimo że dane pochodziły z dwóch odrębnych, niepowiązanych ze sobą bezpośrednio dokumentów, model zidentyfikował miasto **Kraków** jako wspólny mianownik i punkt referencyjny dla obu systemów transportowych.

W strukturze JSON utworzony został jeden, centralny węzeł kotwiczący, do którego skierowano relacje **ŁĄCZY_SIE_Z** zarówno z końca odcinka małopolskiego A4 (węzeł Balice), jak i z początku Zakopianki (odcinek Kraków-Myślenice). Taka topologia w bazie Neo4j umożliwia wykonywanie zaawansowanych algorytmów przeszukiwania grafu (np. najkrótszej ścieżki Dijkstry) w skali całego regionu.

B. Anomalie Semantyczne i Problem "Wysp Danych"

Podczas Próby nr 2 odnotowano zjawisko rozproszenia semantycznego, w którym model stracił zdolność abstrakcji encji nadrzędnych. Zamiast wyodrębnić miasto Kraków jako niezależny obiekt (węzeł), słowo to zostało „uwięzione” wewnątrz ciągów tekstowych opisujących nazwy poszczególnych odcinków (np. „*Zakopianka Kraków - Myślenice*”). Skutkiem tego błędu było powstanie dwóch całkowicie odizolowanych subgrafów, czyli tzw. **wysp danych**. Z punktu widzenia inżynierii ruchu taki graf jest bezużyteczny, ponieważ algorytmy sieciowe interpretują go jako dwa niezależne systemy bez fizycznej możliwości przejazdu między nimi. Zjawisko to dowodzi, że wysokie zagęszczenie informacji w oknie kontekstowym generuje fluktuacje probabilistyczne nawet przy zerowej temperaturze

Metryki skuteczności: Wyniki pokazały rewelacyjną Czułość (Recall) – model nie pominął kluczowych obiektów infrastruktury ujętych w tekstach. Precyzja (Precision) oscylowała w okolicach 76% (średnia ze wszystkich prób), gdyż LLM często realizował tzw. "nad-ekstrakcję" (rozszerzał graf o poprawne logicznie węzły satelickie/kontekstowe, których nie było w ziarnie).

Zjawiska grafowe: Model wykazał zdolność zwaną "Entity Resolution". Przeanalizowano połączone teksty z dwóch dokumentów (A4 oraz DK7). Podczas "Fuzji Wiedzy", AI samoistnie połączyło oba teksty w jedną topologię używając miasta (Kraków) jako węzła kotwiczącego.

Odnotowano jednak również "awarie", gdzie model tworzył dwie niepołączone "wyspy danych", traktując wspólną miejscowość jako zwykły tekst w opisach.

Poniżej zdjęcia (grafy na zdjęciach pochodzą z bazy neo4j)

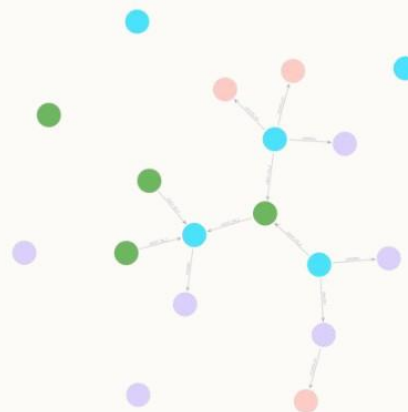
Ziarno:

Case Study: Modernizacja Zakopianki (DK7)

Analiza tekstu technicznego dotyczącego przebudowy układu komunikacyjnego w Krzyszkowicach. Kluczowym wyzwaniem była ekstrakcja relacji między trasą główną (tranzytową) a nowymi drogami lokalnymi.

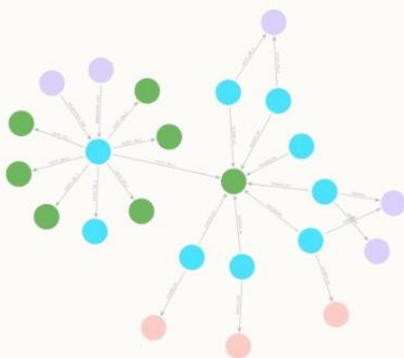
Specyfika Danych:

- ✓ **Obiekty:** Tunel pod DK7, mosty na potokach Krzyszkowianka i Młynówka.
- ✓ **Relacje:** Likwidacja lewoskrętów na rzecz bezkolizyjnego węzła.
- ✓ **Punkty POI:** Przysiółki (Bugaj, Psiary) jako punkty docelowe dróg lokalnych.



Wynik:

Wizualizacja Wyników w Neo4j



Struktura wygenerowanego grafu

Na powyższym schemacie widoczna jest bezpośrednia implementacja wyekstrahowanych danych w bazie Neo4j:

- Węzły centralne:** Reprezentują kluczowe odcinki dróg (np. A4, DK7).
- Obiekty inżynieryjne:** Obiekty inżynieryjne (Wiadukty, Tunele) osadzone na trasie poprzez relację ZAWIERA.
- Skrzyżowania**
- POI**

→ **Topologia:** Strzałki relacji precyzyjnie odwzorowują kierunki potoków ruchu opisane w komunikatach prasowych.

Ekstrakcja Wiedzy: Ziarno vs Próba nr 1

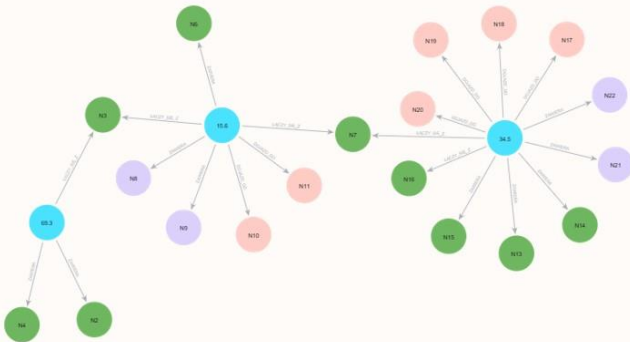
Metryka / Element	Wzorzec (Ziarno)	Wynik LLM (Próba nr 1)	Wniosek
Liczba Węzłów	18 węzłów	24 węzły	Nad-ekstrakcja: model wyodrębnił dodatkowe skrzyżowania (Mogilany, Głogoczów).
Klasyfikacja Dróg	"GP" dla DK7	"DK" oraz "lokalna"	Model wprowadził semantyczny podział klas dróg, którego nie było w ziarnie.
Typy Relacji	ŁĄCZY_SIĘ_Z	DOJAZD_DO	Różnica logiczna: LLM zinterpretował połączenie drogi z węzłem jako funkcję dojazdu.
Relacje Przestrzenne	ZAWIERA (Obiekty)	PRZECHODZI_NAD	Sukces: Model poprawnie zidentyfikował bezkolizyjność kładek dla pieszych.

Obserwacja kluczowa: Próba nr 1 wykazuje tendencję do "wzbogacania" grafu o dodatkowe punkty węzłowe wspomniane w tekście pobocznie (tzw. węzły satelickie), co zwiększa ziarnistość danych kosztem precyzji względem surowego wzorca.

Ziarno:

Case Study 2: Rozbudowa Autostrady A4

Analiza objęła blisko 120-kilometrowy odcinek autostrady A4 w województwach śląskim i małopolskim. Był to najbardziej wymagający tekst pod kątem gęstości informacji technicznych.



Wynik:

Case Study: Rozbudowa Autostrady A4

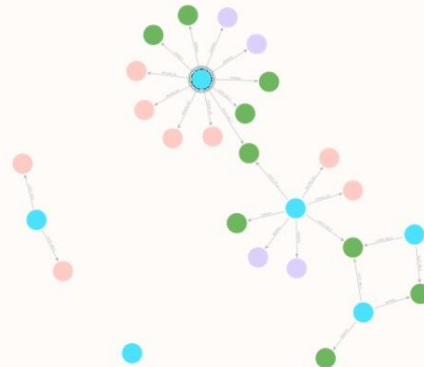
Złożoność Ekstrakcji Wiedzy

Eksperyment na danych A4 wykazał najwyższą "inteligencję topologiczną" modelu LLM. System nie tylko przeniósł dane, ale zaproponował własną strukturę logiczną.

Odkrycie Hierarchii: Model zidentyfikował węzeł nadrzędny "Autostrada A4" (669 km), spinający wszystkie pod-odcinki.

Kontekst Miejski: Samodzielna ekstrakcja miast (Katowice, Kraków) jako punktów docelowych/węzłowych.

Relacje Inkluzji: Poprawne zastosowanie relacji ZAWIERA dla MOP-ów i mostów.



Analiza Porównawcza: Ziarno vs Próba (A4)

Cecha Strukturalna	Wzorzec (Ziarno)	Wynik LLM (Próba nr 1)	Komentarz Badawczy
Liczba Węzłów	22 węzły	27 węzłów	Zwiększona ziarnistość przez dodanie miast i węzła głównego.
Topologia	Plaska (lista odcinków)	Hierarchiczna (Rodzic-Dziecko)	Model LLM poprawnie zdefiniował A4 jako "kontener" dla mniejszych dróg.
Kompletność (Recall)	100%	Zero pominieć (False Negatives) w kluczowych mostach i MOP-ach.	
Relacje POI	DOJAZD_DO	ZAWIERA / DOJAZD_DO	Model użył relacji zawierania dla MOP-ów, co jest technicznie zasadne.

Wniosek: Różnica między Ziarnem a Próbą wynika z **pozytywnej nad-ekstrakcji**. Model LLM "rozumie" infrastrukturę lepiej niż surowy algorytm, uzupełniając brakujące powiązania logiczne między regionami a drogą.

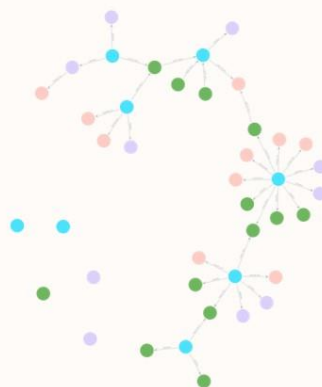
Ziarno:

Fuzja Wiedzy: Eksperyment Zintegrowany (A4 + DK7)

Koncepcja „Super-Ziarna”

Przetworzenie połączonych komunikatów technicznych w ramach jednego okna kontekstowego (Single-Context Window) w celu uzyskania spójnej sieci regionalnej.

-  **Rezolucja Identyfikatorów:** Przesunięcie sekwencyjne ID (A4: N1-N22, Zakopianka: N24-N41) w celu uniknięcia kolizji w bazie danych.
-  **Podejście Single-Context:** Pozwala modelowi LLM na jednoczesny dostęp do obu tekstów, co jest kluczem do odnalezienia punktów wspólnych.



Wynik:

Próba nr 1: Sukces Rezolucji Encji i Fuzji – Kraków jako „Anchor Node”

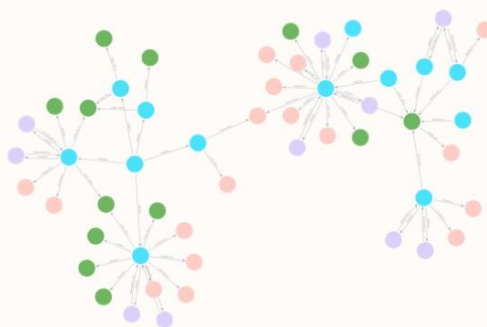
Prawidłowa Abstrakcja Geograficzna

Model w tej próbie zachował się jak rasowy inżynier GIS – dostrzegł, że „Kraków” to punkt centralny dla obu systemów drogowych.

-  **Utworzenie Węzła Centralnego:** Model wyodrębnił węzeł N26 (Miasto: Kraków).
-  **Kotwiczenie Autostrady A4:** Odcinek Katowice–Kraków (N24) został połączony bezpośrednią relacją ŁĄCZY_SIE_Z z Krakowem (N26).
-  **Kotwiczenie Zakopianki:** Odcinek DK7 Kraków–Myślenice (N40) również został połączony relacją ŁĄCZY_SIE_Z z tym samym Krakowem (N26).

Efekt w bazie Neo4j:

Dzięki spięciu obu dróg przez wspólny punkt referencyjny (Węzeł N26), algorytmy przeszukiwania grafu (np. Dijkstra/A*) mogą bez przeszkód wyznaczać trasy tranzytowe z Katowic prosto w Tatry. Fuzja zakończyła się pełnym sukcesem topologicznym.



Wynik 2:

Próba nr 2: Awaria Fuzji ("Wyspy Danych")

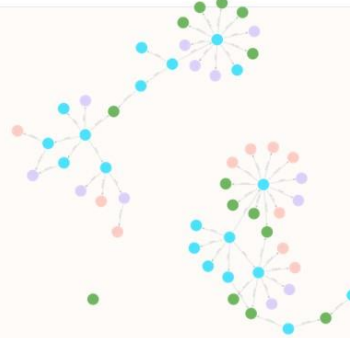
Błąd Semantycznego Rozproszenia

W tej próbie model stracił zdolność abstrakcji encji nadrzędnych. Zamiast stworzyć jeden węzeł dla miasta Kraków, zamknął informację geograficzną wewnątrz stringów.

- ❌ **Brak Węzła Wspólnego:** W spisie 55 węzłów nie istnieje dedykowany obiekt dla Krakowa ani Balic (Balice pojawiają się tylko jako Punkt POI N28, ale nie są połączone z DK7).
- ⚠️ **Informacja jako Tekst, nie Obiekt:** Słowo "Kraków" zostało uwiecznione w nazwach odcinków: N25 ("A4 Katowice – Kraków"), N27 ("A4 Katowice – Balice"), N32 ("Zakopianka Kraków – Mysłenice").

Skutek dla bazy danych:

Analiza relacji w Próbie nr 2 wykazuje **całkowity brak połączenia** pomiędzy podsiecią A4 a podsiecią DK7. Powstały dwie całkowicie odizolowane "wyspy danych". System komputerowy analizujący ten graf uznałby, że z autostrady A4 nie da się zjechać na Zakopiankę, co eliminuje ten graf z użytku inżynierskiego.

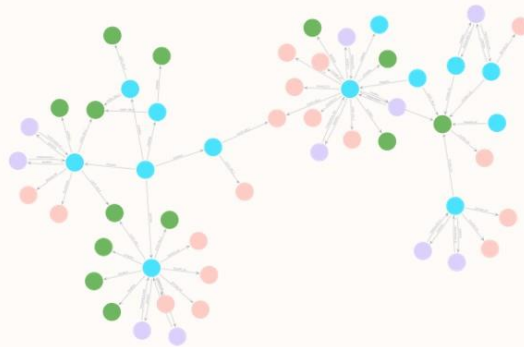


Fuzja Wiedzy i Entity Resolution

Integracja Wieloźródłowa

Największym sukcesem projektu była zdolność modelu do autonomicznego łączenia grafów z różnych dokumentów (A4 oraz DK7).

- 📌 **Węzeł Kotwiczący:** Automatyczne rozpoznanie "Krakowa" jako wspólnego punktu referencyjnego.
- 🔗 **Scalenie Topologii:** Połączenie końca Autostrady A4 z początkiem Zakopianki w jedną sieć.
- 📊 **Rezultat:** Przejście z odizolowanych "wysp danych" do zintegrowanego grafu transportowego regionu.



Ewaluacja Fuzji Wiedzy: Ziarno vs Próba 1 i 2

Cecha Strukturalna	Wzorzec (Ziarno Małopolski)	Próba nr 1 (Sukces Fuzji)	Próba nr 2 (Anomalia Strukturalna)
Liczba Węzłów	41 węzłów	53 węzły (Nad-ekstrakcja kontekstowa)	55 węzłów (Nadmierna segmentacja)
Encja Kotwicząca (Kraków)	Wyodrębniona jako osobny węzeł POI (N23)	Wyodrębniona jako osobny węzeł Miasto (N26)	Brak węzła. Rozproszona w nazwach właściwości odcinków.
Topologia Sieci	Zintegrowany graf (Jedna sieć)	Połączona spójnie	Rozbita (2 odizolowane subgrafy)
Relacje Nadrzędne	Płaskie mapowanie ciągów	A4 zdefiniowana jako kontener makro (N1)	A4 zdefiniowana jako kontener makro (N1)

Wniosek metodologiczny: Mimo identycznego promptu i reżimu temperaturowego ($T=0.0$), wysokie zagęszczenie informacji w tekście zintegrowanym wywołało fluktuację probabilistyczną, prowadząc do dwóch skrajnie różnych strategii mapowania świata przez LLM.

Analiza Anomalii: Tokenization Noise

Zjawisko błędu syntaktycznego

Eksperymenty w trybie stochastycznym ujawniły błędy tokenizacji, które uniemożliwiają automatyczny import do bazy Neo4j.

```
"source": "N3",
"target": "N9",
"type": "ŁĄCZY_SIEĆ_Z"
```

Błąd: Nadmiarowy token "I" w nazwie relacji.

Rozwiązanie: Wdrożenie warstwy walidacji **Pydantic** wymuszającej zgodność ze schematem przed importem.

1. Czułość (Recall) – Pełność Ekstrakcji

Matematycznie metryka ta definiowana jest jako stosunek poprawnie wyekstrahowanych elementów do wszystkich elementów, które realnie znajdowały się w tekście źródłowym:

Wartość inżynierska: W systemach automatycznego przetwarzania danych technicznych, czułość jest kluczowym wskaźnikiem bezpieczeństwa i integralności informacji. Mierzy ona zdolność algorytmu do bezbłędneho wykrywania wszystkich obiektów rzeczywistych opisanych w tekście źródłowym, dążąc do zminimalizowania liczby pominięć (False Negatives). Z punktu widzenia inżynierii drogowej wysoki wskaźnik Recall jest absolutnym priorytetem – gwarantuje on, że podczas automatycznego zasilania bazy danych żaden strategiczny obiekt infrastruktury (taki jak most, wiadukt, tunel czy MOP) nie zostanie przeoczony. W systemach ekspertowych metryka ta decyduje o kompletności wiedzy i

eliminuje ryzyko powstawania krytycznych luk informacyjnych w cyfrowym modelu sieci drogowej.

2. Precyzja (Precision) – Zaufanie do Wygenerowanych Encji

Precyzja określa, jaki procent spośród wszystkich obiektów wskazanych przez model LLM stanowiły elementy poprawne i oczekiwane według Złotego Standardu:

Wartość inżynierska: W systemach ekstrakcji informacji z tekstu naturalnego, precyzja określa stopień czystości i dokładności generowanych danych. Jej zadaniem jest kontrolowanie liczby obiektów nadmiarowych (False Positives) w stosunku do bazy wzorcowej. W kontekście automatycznego budowania grafów wiedzy dla infrastruktury, spadki tej metryki nie muszą jednak oznaczać błędów krytycznych czy halucynacji modelu LLM. Bardzo często są one efektem tzw. pozytywnej nad-ekstrakcji. Model językowy, przetwarzając tekst, identyfikuje i mapuje poprawne logicznie węzły kontekstowe (np. nazwy powiązanych miejscowości, regionów czy tła sieciowego), które pominięto w uproszczonym wzorcu ludzkim. Z punktu widzenia inżynierii wiedzy, zjawisko to podnosi wartość końcowej bazy danych, zwiększając jej ziarnistość oraz dostarczając szerszego kontekstu relacyjnego..

3. F1-Score – Harmoniczna Średnia Skuteczności

Miara F1 reprezentuje zrównoważony bilans pomiędzy Precyzją a Czułością. Wykorzystuje średnią harmoniczną, co oznacza, że drastyczny spadek jednej z metryk skrajnie obniżyłby wynik końcowy:

Wartość inżynierska: Miara F1 stanowi zintegrowany wskaźnik efektywności systemów ekstrakcji informacji, będący średnią harmoniczną czułości oraz precyzji. W inżynierii wiedzy metryka ta pełni kluczową rolę oceny systemowej – zapobiega sytuacji, w której wysoki wynik jednej składowej maskuje drastyczne błędy drugiej. Wysoki wskaźnik F1-Score potwierdza, że automatyczny potok danych zachowuje optymalny kompromis projektowy. Oznacza to, że system z powodzeniem realizuje dwa sprzeczne cele: gwarantuje bezwzględną kompletność danych technicznych (wysoki *Recall*), a jednocześnie utrzymuje pod ścisłą kontrolą wolumen nadmiarowych informacji bocznych (stabilne *Precision*), chroniąc końcowy graf przed chaosem strukturalnym.

4. Indeks Jaccarda – Współczynnik Podobieństwa Grafów

Indeks Jaccarda określa stopień podobieństwa między dwoma zbiorami. Oblicza się go jako stosunek wielkości części wspólnej (obiektów identycznych w obu zbiorach) do wielkości ich sumy (wszystkich unikalnych obiektów wygenerowanych i wzorcowych).

Wartość inżynierska: W systemach analizy grafów wiedzy oraz hurtowniach danych, Indeks Jaccarda służy do globalnej oceny spójności strukturalnej. Podczas gdy Precision i Recall oceniają wyciąganie pojedynczych obiektów, Jaccard patrzy na graf jako na całą przestrzeń sieciową. W kontekście inżynierii infrastruktury liniowej, wysoka wartość tego indeksu daje projektantowi gwarancję, że cyfrowa mapa stworzona przez model LLM jest kalką logiczną założeń projektowych. Z kolei gwałtowny spadek Indeksu Jaccarda to najszybszy sygnał ostrzegawczy o awarii topologii grafu – informuje on inżyniera, że w systemie doszło do poważnych błędów (np. rozbicia sieci na odizolowane „wyspy danych”, nadmiernego poszatkwania odcinków lub pominięcia kluczowych punktów zwrotnych, takich jak węzły miejskie). Metryka ta pozwala na natychmiastowe odróżnienie grafu w pełni użytecznego technicznie od struktury wadliwej.

Wzory:

Recall (Pełność / Czulość)

Odpowiada na pytanie: "Jaki procent rzeczywistych obiektów z tekstu udało się modelowi odnaleźć?"

Wzór matematyczny:

$$\text{Recall} = \frac{TP}{TP + FN}$$

W naszym projekcie Recall = 100%, co oznacza całkowity brak pominąć obiektów krytycznych (mosty, wiadukty).

Precision (Precyzja)

Odpowiada na pytanie: "Ile z obiektów wygenerowanych przez model faktycznie powinno znaleźć się w grafie?"

Wzór matematyczny:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Niższa precyzja (~77%) wynika z dodawania przez LLM węzłów kontekstowych (miasta, regiony), których nie było we wzorcowym ziarnie.

F1-Score

Średnia harmoniczna precyzji i pełności. Jest to najbardziej obiektywna miara skuteczności modelu.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Wynik **87.2%** potwierdza bardzo wysoką sprawność modelu w zadaniach inżynierii danych.

Indeks Jaccarda (Similarity)

Mierzy podobieństwo dwóch zbiorów (Wzorzec vs Wynik AI). Oblicza stosunek części wspólnej do sumy zbiorów.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Wartość **0.75** wskazuje na silną zbieżność topologiczną wygenerowanego grafu z rzeczywistym układem drogowym.

Obliczenia:

Dla samej Autostrady A4:

- Ziarno = 22 węzły, LLM = 27 węzłów
- Recall = 100% (22 z 22)
- Precision = 81.48%
- F1-Score = 89.8%
- Indeks Jaccarda = 81.48%

Dla samej Zakopianki DK7:

- Ziarno = 18 węzłów, LLM = 24 węzły
- Recall = 100% (18 z 18)
- Precision = 75.0%
- F1-Score = 85.7%
- Indeks Jaccarda = 75%

Dla połączonych tekstów:

Scenariusz A: Próba nr 1 i 3 (Sukces Fuzji)

- Ziarno = 41 węzłów, LLM = 53 węzły
- Recall = 100%
- Precision = 77.36%
- F1-Score = 87.24%
- Indeks Jaccarda = 77.36%

Scenariusz B: Próba nr 2 (Wyspy danych)

- Ziarno = 41 węzłów, LLM = 55 węzły
- Recall = 95.12%
- Precision = 70.91%
- F1-Score = 81.25%

- Indeks Jaccarda = 68.42%

Podsumowanie / Źródła

Osiągnięcia: Osiągnięto gigantyczną przewagę kosztową i czasową, skracając analizę danych z godzin do sekund. Wskazano, że graf Neo4j jest naturalnie lepszy niż zwykłe relacyjne bazy tabelaryczne do mapowania sieci topologicznej.

Rekomendacje na przyszłość (Dalsze prace): Jako optymalne wdrożenie zalecono wykorzystanie zasady "Human-in-the-Loop", by wygenerowany przez maszynę graf-szkic sprawdzał inżynier. Rekomenduje się również migrację z płatnych API (Gemini/OpenAI) na otwarte modele instalowane lokalnie (np. Llama-3, Mistral), co zapewni darmowe operacje i chroni wrażliwe dane przetargowe GDDKiA.

Plusy (Silne Strony) Zaproponowanej Metody

Efektywność i Skalowalność

- ⚡ **Radikalna automatyzacja:** Skrócenie czasu ekstrakcji danych z dokumentacji technicznej z godzin (praca manualna) do sekund.
- 🔄 **Przetwarzanie masowe:** Możliwość równoległej analizy setek komunikatów i raportów bez wzrostu zapotrzebowania na zasoby ludzkie.

Głęboka Semantyczna i Fuzja

- 🔍 **Recall na poziomie 100%:** Gwarancja, że żaden kluczowy obiekt infrastruktury wymieniony w tekście nie zostanie pominięty.
- 🧩 **Autonomiczne Entity Resolution:** Zdolność do samodzielnego łączenia niezależnych dokumentów w jedną logiczną sieć grafową (np. wokół węzłów miejskich).

Minusy (Ograniczenia i Słabe Strony)

Problemy Generatywne i Składniowe

- ⚠️ **Szum Tokenizacji (T>O.O):** Ryzyko powstawania błędów w nazwach relacji (np. "ŁĄCZY_SIEĆ_Z"), blokujących import do bazy.
- 🔴 **Nad-ekstrakcja (False Positives):** Tworzenie nadmiarowych, ogólnych węzłów kontekstowych (np. regionów), co obniża precyzję.

Zależności Technologiczne

- 🔧 **Wrażliwość na Prompting:** Wynik końcowy jest silnie uzależniony od precyzyjnej konstrukcji instrukcji systemowych i przykładów Few-Shot.
- 💰 **Koszty utrzymania:** Komercyjne API (OpenAI/Gemini) generują koszty proporcjonalne do wolumenu tokenów przy skomplikowanych promptach.

Kluczowe Wnioski Badawcze i Metodologiczne

- Prymat Pełnego Determinizmu ($T=0.0$):** Dla systemów zasilających relacyjne lub grafowe bazy danych, temperatura zero jest warunkiem koniecznym do zachowania spójności schematu.
- Niezbędność Walidacji Hybrydowej:** Sam model LLM nie jest systemem autonomicznym. Konieczne jest stosowanie zewnętrznych parserów (np. Pydantic, JSON Schema) jako "bezpieczników" strukturalnych.
- Graf jako Optymalny Model Świata:** Baza Neo4j idealnie odzwierciedla nieliniową i wielopoziomą naturę opisów infrastruktury liniowej, przewyższając klasyczne tabele relacyjne.

Rekomendacje dla Wdrożeń Przemysłowych

Architektura Systemu

Podjęcie Human-in-the-Loop: System powinien generować graf jako "wersję kandydacką" (wstępną), która podlega szybkiej, wizualnej akceptacji przez inżyniera kontraktu, eliminując błędy typu False Positive.

Optymalizacja i Bezpieczeństwo

Lokalne Modele LLM (Open-Source): W celu redukcji kosztów API oraz ochrony danych wrażliwych (np. niepubliczne plany przetargowe), rekomenduje się migrację na wyspecjalizowane, lokalne modele klasy Llama-3 lub Mistral dostosowane do języka polskiego.

Opracowany prototyp systemu udowodnił, że automatyczna transformacja nieustrukturyzowanych opisów drogowych na bazy grafowe jest w pełni możliwa i wysoce wydajna. Aby jednak przekształcić to rozwiązanie w system klasy produkcyjnej (gotowy do wdrożenia w instytucjach takich jak GDDKiA), w przyszłości należy skupić się na rozwoju następujących obszarów:

Migracja na lokalne modele otwartoźródłowe (Open-Source LLM)

Obecna architektura wykorzystuje komercyjne API Google Gemini. Choć zapewnia ono doskonałą jakość, wiąże się z kosztami subskrypcyjnymi oraz koniecznością wysyłania danych do chmury. Kolejnym krokiem rozwojowym powinna być migracja systemu na zaawansowane modele lokalne, instalowane na własnej infrastrukturze sprzętowej (np. **Llama 3 70B** lub **Mistral**). Pozwoli to na:

- Całkowite uniezależnienie się od opłat za zapytania (tokeny).
- Gwarancję stuprocentowej prywatności danych – pliki przetargowe i plany modernizacji nie opuszczają serwerów wewnętrznych urzędu.

Rozbudowa modułu walidacji do trybu aktywnej samo-naprawy (Self-Correction)

Obecny moduł walidacji (VALID_ONTOLOGY) działa w trybie pasywnego audytu – wykrywa błędy modelu Gemini i informuje o nich użytkownika w logach. Przyszłe prace powinny dążyć do wdrożenia mechanizmu **aktywnej samo-naprawy**. W momencie, gdy skrypt Pythona wykryje relację łamiącą zasady ontologii (np. połączenie odcinka drogi bezpośrednio z mostem poprzez relację ŁĄCZY_SIE_Z), system powinien automatycznie wysłać do LLM zapytanie korygujące: „*Popełniłeś błąd semantyczny w relacji X. Popraw to zgodnie ze schematem*”. Pozwoli to na automatyczne oczyszczanie grafu przed zapisem, bez konieczności interwencji człowieka.

Bibliografia / Źródła:

- [1] Google Developer Library, *Gemini API Overview: Structured Outputs with Response MIME-types*, Google AI Documentation, URL: <https://ai.google.dev/gemini-api/docs/structured-output?example=recipe>
- [2] Neo4j Drivers Manual v5.0, *Python Driver Overview: Establishing Secure Connections and Cypher Query Execution*, Neo4j Documentation, URL: <https://neo4j.com/docs/python-manual/current/>
- [3] Neo4j Cypher Manual v5.0, *Clauses: MERGE, MATCH, and Relationship Constraints*, Neo4j Documentation, URL: <https://neo4j.com/docs/cypher-manual/current/>
- [4] Python Software Foundation, *Standard Library: json — JSON encoder and decoder*, Python Documentation v3.10. URL: <https://docs.python.org/3/library/json.html>
- [5] Generalna Dyrekcja Dróg Krajowych i Autostrad, *Kompleksowe zmiany na A4*, gov.pl, 2024. URL: <https://www.gov.pl/web/gddkia/kompleksowe-zmiany-na-a4>. (Oficjalne źródło danych dla analizy semantycznej Autostrady A4).
- [6] Generalna Dyrekcja Dróg Krajowych i Autostrad - Oddział Kraków, *Węzeł krajowy, drogi lokalne – Program Inwestycji dla Krzyszkowic na DK7*, gov.pl. URL: <https://www.gov.pl/web/gddkia-krakow/wezel-krajowy-drogi-lokalne>. (Oficjalne źródło danych dla analizy semantycznej Zakopianki).

Technologie: Python, Neo4j Aura, Cypher, Google Gemini Flash.

Kod:

```
import json
import os
from google import genai
from google.genai import types
from neo4j import GraphDatabase
from dotenv import load_dotenv

load_dotenv()

REQUIRED_ENV_VARS = ["NEO4J_URI", "NEO4J_USER", "NEO4J_PASSWORD",
"GENEIMI_API_KEY"]
```

```

missing_vars = [var for var in REQUIRED_ENV_VARS if not os.getenv(var)]

if missing_vars:
    print("\n[BŁĄD KRYTYCZNY] Brak wymaganych zmiennych środowiskowych w
    pliku .env!")
    print(f"Uzupełnij w pliku .env następujące pola: {'',
    '.join(missing_vars)}")
    print("Skopiuj plik .env.example jako .env i wpisz poprawne dane.")
    exit(1)

# --- KONFIGURACJA ---
NEO4J_URI = os.getenv("NEO4J_URI")
NEO4J_USER = os.getenv("NEO4J_USER")
NEO4J_PASSWORD = os.getenv("NEO4J_PASSWORD")
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")

# Inicjalizacja nowego klienta Gemini
client = genai.Client(api_key=GEMINI_API_KEY)

SYSTEM_PROMPT = """
Jesteś ekspertem w dziedzinie inżynierii drogowej i analizy danych
grafowych. Twoim zadaniem
jest przekształcanie tekstowych opisów infrastruktury liniowej na
ustrukturyzowany format
JSON, który zostanie zaimportowany do bazy Neo4j.
1. DOPUSZCZALNE WĘZŁY I ATRYBUTY:
• Odcinek drogi: {nazwa, nawierzchnia, dlugosc, liczba_pasow, klasa}
• Skrzyżowanie: {nazwa, rodzaj, sygnalizacja (Tak/Nie)}
• Rondo: {nazwa, liczba_wlotow, lokalizacja}
• Obiekt inżynieryjny: {typ_obiektu (Most/Wiadukt/Tunel/Przepust), nazwa,
nosnosc}
• Punkt POI: {kategoria, nazwa_wlasna}
2. DOPUSZCZALNE RELACJE:
• ŁĄCZY_SIĘ_Z: (Odcinek <-> Skrzyżowanie/Rondo)
• ZAWIERA: (Odcinek -> Obiekt inżynieryjny/Rondo)
• PRZECHODZI_NAD / PRZECHODZI_POD: (Obiekt inżynieryjny <-> Odcinek
drogi)
• DOJAZD_DO: (Odcinek drogi/Obiekt inżynieryjny -> Punkt POI)
3. ZASADY EKSTRAKCJI:
• Zwracaj wyniki WYŁĄCZNIE w formacie JSON.
• Każdy węzeł musi mieć unikalne ID (np. N1, N2).
• Jeśli brakuje jakiegoś atrybutu w tekście, pomij go w obiekcie
properties.
4. PRZYKŁADY (Few-Shot):
{
  "document_info": {
    "title": "Obwodnica Przecławia i Warzymic (DK13)",
    "date": "2023-08-31",
    "source_type": "Real-world technical description"
  },
  "nodes": [
    {
      "id": "N1",
      "type": "Odcinek drogi",
      "properties": {
        "nazwa": "Obwodnica Przecławia i Warzymic - Etap I",
        "dlugosc": 4.2,

```

```
"liczba_pasow": 4,
"klasa": "GP"
},
{
  "id": "N2",
  "type": "Odcinek drogi",
  "properties": {
    "nazwa": "Obwodnica Przecławia i Warzymic - Etap II",
    "dlugosc": 2.3,
    "klasa": "S/GP"
  }
},
{
  "id": "N3",
  "type": "Odcinek drogi",
  "properties": {
    "nazwa": "Obwodnica Kołbaskowa",
    "dlugosc": 5.6,
    "liczba_pasow": 2,
    "klasa": "GP"
  }
},
{
  "id": "N4",
  "type": "Odcinek drogi",
  "properties": {
    "nazwa": "Autostrada A6"
  }
},
{
  "id": "N5",
  "type": "Odcinek drogi",
  "properties": {
    "nazwa": "Droga lokalna Kurów - Przecław"
  }
},
{
  "id": "N6",
  "type": "Rondo",
  "properties": {
    "nazwa": "Rondo Hakena",
    "lokalizacja": "Szczecin"
  }
},
{
  "id": "N7",
  "type": "Rondo",
  "properties": {
    "nazwa": "Rondo południowe",
    "lokalizacja": "styk z DK13 na południe od Przecławia"
  }
},
{
  "id": "N8",
  "type": "Rondo",
  "properties": {
```

```
"nazwa": "Rondo Kołbaskowo-Moczyły"
},
{
  "id": "N9",
  "type": "Rondo",
  "properties": {
    "nazwa": "Rondo Rosówek"
  }
},
{
  "id": "N10",
  "type": "Rondo",
  "properties": {
    "nazwa": "Rondo graniczne",
    "lokalizacja": "Rosówek - Kamieniec"
  }
},
{
  "id": "N11",
  "type": "Skrzyżowanie",
  "properties": {
    "nazwa": "Węzeł Przecław",
    "rodzaj": "węzeł drogowy"
  }
},
{
  "id": "N12",
  "type": "Skrzyżowanie",
  "properties": {
    "nazwa": "Węzeł Siadło Górne",
    "rodzaj": "węzeł drogowy"
  }
},
{
  "id": "N13",
  "type": "Skrzyżowanie",
  "properties": {
    "nazwa": "Węzeł Szczecin Zachód",
    "rodzaj": "węzeł autostradowy"
  }
},
{
  "id": "N14",
  "type": "Obiekt inżynierski",
  "properties": {
    "typ_obiektu": "Wiadukt",
    "nazwa": "Wiadukt na węźle Przecław"
  }
},
{
  "id": "N15",
  "type": "Obiekt inżynierski",
  "properties": {
    "typ_obiektu": "Wiadukt",
    "nazwa": "Wiadukt w ciągu drogi Kurów - Przecław"
  }
}
```

```

},
{
  "id": "N16",
  "type": "Obiekt inżynierski",
  "properties": {
    "typ_obiektu": "Przepust",
    "nazwa": "Przepusty na obwodnicy"
  }
}
],
"relationships": [
  { "source": "N1", "target": "N6", "type": "ŁĄCZY_SIE_Z" },
  { "source": "N1", "target": "N7", "type": "ŁĄCZY_SIE_Z" },
  { "source": "N1", "target": "N11", "type": "ŁĄCZY_SIE_Z" },
  { "source": "N1", "target": "N16", "type": "ZAWIERA" },
  { "source": "N14", "target": "N1", "type": "PRZECHODZI_NAD" },
  { "source": "N15", "target": "N1", "type": "PRZECHODZI_NAD" },
  { "source": "N1", "target": "N12", "type": "ŁĄCZY_SIE_Z" },
  { "source": "N2", "target": "N12", "type": "ŁĄCZY_SIE_Z" },
  { "source": "N2", "target": "N13", "type": "ŁĄCZY_SIE_Z" },
  { "source": "N4", "target": "N13", "type": "ŁĄCZY_SIE_Z" },
  { "source": "N3", "target": "N13", "type": "ŁĄCZY_SIE_Z" },
  { "source": "N3", "target": "N8", "type": "ZAWIERA" },
  { "source": "N3", "target": "N9", "type": "ZAWIERA" },
  { "source": "N3", "target": "N10", "type": "ZAWIERA" }
]
}

```

ZADANIE: Przeanalizuj poniższy tekst i wygeneruj graf JSON zgodnie z powyższym schematem:

```

"""

def extract_with_gemini(raw_text, filename):
    print("Gemini analizuje tekst z parametrem (temperature=0.0)...")
    model_id = "gemini-2.5-flash"

    try:
        config = types.GenerateContentConfig(
            temperature=0.0,
            response_mime_type="application/json"
        )

        response = client.models.generate_content(
            model=model_id,
            contents=f"{SYSTEM_PROMPT}\n\nTekst do analizy: {raw_text}",
            config=config
        )

        text_response = response.text
        clean_json = text_response.replace('```json', '').replace('```',
            '').strip()

        start_idx = clean_json.find('{')
        end_idx = clean_json.rfind('}') + 1
        final_json_str = clean_json[start_idx:end_idx]

        parsed_json = json.loads(final_json_str)

```

```

print("\n" + "=" * 50)
print("WYGENEROWANY STABILNY JSON:")
print(json.dumps(parsed_json, indent=4, ensure_ascii=False))
print("=" * 50 + "\n")

output_filename = filename.replace('.txt', '_wynik.json')

with open(output_filename, "w", encoding="utf-8") as f:
    json.dump(parsed_json, f, indent=4, ensure_ascii=False)
print(f"Wynik zapisanego pliku: {output_filename}")

return parsed_json

except Exception as e:
    print(f"Błąd podczas komunikacji z modelem {model_id}: {e}")
    raise e

def save_to_neo4j(data):
    print("Rozpocynam miękka walidację schematu i przesyłanie do Neo4j...")

    node_type_map = {node['id']: node['type'] for node in
data.get('nodes', [])}

    VALID_ONTOLOGY = {
        "ŁĄCZY_SIE_Z": {
            ("Odcinek drogi", "Skrzyżowanie"), ("Skrzyżowanie", "Odcinek
drogi"),
            ("Odcinek drogi", "Rondo"), ("Rondo", "Odcinek drogi")
        },
        "ZAWIERA": {
            ("Odcinek drogi", "Obiekt inżynieryjny"),
            ("Odcinek drogi", "Rondo")
        },
        "PRZECHODZI_NAD": {
            ("Obiekt inżynieryjny", "Odcinek drogi"),
            ("Odcinek drogi", "Obiekt inżynieryjny")
        },
        "PRZECHODZI_POD": {
            ("Obiekt inżynieryjny", "Odcinek drogi"),
            ("Odcinek drogi", "Obiekt inżynieryjny")
        },
        "DOJAZD_DO": {
            ("Odcinek drogi", "Punkt POI"),
            ("Obiekt inżynieryjny", "Punkt POI")
        }
    }

    driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER,
NEO4J_PASSWORD))
    with driver.session() as session:

        for node in data.get('nodes', []):
            query = f"MERGE (n:`{node['type']}` `{{id: $id}}`) SET n +=
$props"

```

```

        session.run(query, id=node['id'], props=node['properties'])

    for rel in data.get('relationships', []):
        source_id = rel['source']
        target_id = rel['target']
        rel_type = rel['type']

        is_valid = True
        reason = ""

        if source_id not in node_type_map or target_id not in
node_type_map:
            is_valid = False
            reason = "Brak definicji jednego z węzłów w sekcji
'nodes'."
        else:
            source_type = node_type_map[source_id]
            target_type = node_type_map[target_id]

            if rel_type not in VALID_ONTOLOGY:
                is_valid = False
                reason = f"Nieznany typ relacji '{rel_type}' poza
oficjalną specyfikacją."
            else:
                current_pair = (source_type, target_type)
                if current_pair not in VALID_ONTOLOGY[rel_type]:
                    is_valid = False
                    reason = f"Niedozwolone połączenie typów:
({source_type}) -[{rel_type}]-> ({target_type})."

            if not is_valid:
                print(f"WALIDACJA OSTRZEGAWCZA: Model utworzył relację
niezgodną z ontologią!")
                print(f"    Połączenie: {source_id} -[{rel_type}]->
{target_id} | Powód: {reason}")
                print(f"    [INFO] Relacja zostaje mimo to wymuszona i
zapisana w bazie Neo4j.")

            query = f"MATCH (a {{id: $source}}), (b {{id: $target}})
MERGE (a)-[:{rel_type}]->(b)"
            session.run(query, source=source_id, target=target_id)

    driver.close()
    print("Graf zaktualizowany (zastosowano asynchroniczny audyt
ontologiczny)!")

if __name__ == "__main__":
    DATA_FOLDER = "data"

    if not os.path.exists(DATA_FOLDER):
        os.makedirs(DATA_FOLDER)
        print(f"Stworzono folder '{DATA_FOLDER}'. Wrzuć tam pliki
tekstowe (.txt) i uruchom program ponownie.")
        exit()

    pliki = [f for f in os.listdir(DATA_FOLDER) if f.endswith('.txt')]

```

```

    if not pliki:
        print(f"Folder '{DATA_FOLDER}' jest pusty. Brak tekstów do
wyświetlenia.")
        exit()

    print("\n" + "=" * 15 + " MENU WYBORU PLIKÓW " + "=" * 15)
    for i, plik in enumerate(pliki, 1):
        print(f" [{i}] -> {plik}")
    print("=" * 50)

    wybor_user = input("Wpisz numery plików do przetworzenia oddzielone
spacją (np. 1 2 5): ")

    wybrane_indeksy = []
    for element in wybor_user.split():
        if element.isdigit():
            indeks = int(element) - 1
            if 0 <= indeks < len(pliki):
                wybrane_indeksy.append(indeks)

    if not wybrane_indeksy:
        print("Nie wybrano żadnego poprawnego pliku. Koniec programu.")
        exit()

    print(f"\nRozpaczynam sekwencyjne przetwarzanie wybranych plików
({len(wybrane_indeksy)})...")

    sukcesy = []
    bledy = []

    for idx in wybrane_indeksy:
        nazwa_pliku = pliki[idx]
        pelna_sciezka = os.path.join(DATA_FOLDER, nazwa_pliku)

        print("\n" + "#" * 30)
        print(f"PRZETWARZAM PLIK: {nazwa_pliku}")
        print("#" * 30)

        try:
            with open(pelna_sciezka, "r", encoding="utf-8") as f:
                tekst_z_pliku = f.read()

                wynik_json = extract_with_gemini(tekst_z_pliku, nazwa_pliku)
                save_to_neo4j(wynik_json)

                print(f"Sukces dla pliku: {nazwa_pliku}")
                sukcesy.append(nazwa_pliku)

        except Exception as e:
            print(f"Problem podczas przetwarzania pliku {nazwa_pliku}:
{e}")

            bledy.append((nazwa_pliku, str(e)))
            print("Przechodzę do kolejnego wybranego zadania...")

    print("\n" + "=" * 15 + " PODSUMOWANIE PROCESU " + "=" * 15)
    if sukcesy:

```

```
print(f"Pomyślnie przetworzono pliki ({len(sukcesy)}):")
for s in sukcesy:
    print(f"    - {s}")

if bledy:
    print(f"\nNie udało się przetworzyć plików ({len(bledy)}):")
    for b_plik, b_powod in bledy:
        print(f"    - {b_plik} | Powód: {b_powod}")

print("=" * 52)
```